

Релационен модел на БД

- упражнение

Н. Панайотова



Задача:

Създайте БД с GUI със следните таблици и съответните атрибути в тях:

01

- Продукти – име, вид/категория, доставчик, доставна цена, налично количество, преустановяване на продажбата (да/не).

02

- Служители – име, фамилия, длъжност, ЕГН, дата на постъпване, тел. Номер, адрес, e-mail.

03

- Доставчици – име на фирмата доставчик, лице за контакт, длъжност на лицето, адрес, телефон, e-mail.

04

- Доставки – информация за доставчик, служител, дата на доставка, доставено количество от съответния продукт, цена на доставката.

Table Name: Schema: **магаз**

Charset/Collation: Engine:

Comments:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/E
⚡ ИД на протукт	INT	☒	☒	☐	☐	☐	☐	☐	☐	
◇ Име	VARCHAR(45)	☐	☐	☐	☐	☐	☐	☐	☐	
◇ Вид	VARCHAR(45)	☐	☐	☐	☐	☐	☐	☐	☐	
◇ Доставчик	VARCHAR(45)	☐	☐	☐	☐	☐	☐	☐	☐	
◇ Доставка цена	DECIMAL(3,2)	☐	☐	☐	☐	☐	☐	☐	☐	
◇ Налично количество	INT	☐	☐	☐	☐	☐	☐	☐	☐	
◇ Преустановен	TINYINT	☐	☐	☐	☐	☐	☐	☐	☐	0

DECIMAL(3,2) се използва за пари
където имаме 3 числа пред
десетичната запетая и 2 след нея.

В BD тип **BOOLEAN = TINYINT**

```

1 CREATE TABLE `магазин`.`продукти` (
2   `ИД на протукт` INT NOT NULL,
3   `Име` VARCHAR(45) NULL,
4   `Вид` VARCHAR(45) NULL,
5   `Доставчик` VARCHAR(45) NULL,
6   `Доставна цена` DECIMAL(3,2) NULL,
7   `Налично количество` INT NULL,
8   `Преустановен` TINYINT NULL DEFAULT 0,
9   PRIMARY KEY (`ИД на протукт`));
10

```

ИД на протукт	Име	Вид	Доставчик	Доставна цена	Налично количество	Преустановен
1	Хляб "Оборище"	хляб	Фирма "Удобство"	1.50	10	0
2	Хляб "Зърно"	хляб	Фирма "Бързина"	1.45	15	0
3	Захар "Сладост"	захар	Фирма "Удобство"	3.00	5	0
4	Захар "Тръстикова"	захар	Фирма "Удобство"	3.50	4	1
NULL	Прясно мляко "Пасище"	прясно мляко	Фирма "Удобство"	3.50	8	0
NULL	Прясно мляко "Ливади"	прясно мляко	Фирма "Бързина"	3.45	7	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL

```

1 INSERT INTO `магазин`.`продукти` (`ИД на протукт`, `Име`, `Вид`, `Доставчик`
2 INSERT INTO `магазин`.`продукти` (`ИД на протукт`, `Име`, `Вид`, `Доставчик`
3 INSERT INTO `магазин`.`продукти` (`ИД на протукт`, `Име`, `Вид`, `Доставчик`
4 INSERT INTO `магазин`.`продукти` (`ИД на протукт`, `Име`, `Вид`, `Доставчик`
5 INSERT INTO `магазин`.`продукти` (`Име`, `Вид`, `Доставчик`, `Доставна цена`
6 INSERT INTO `магазин`.`продукти` (`Име`, `Вид`, `Доставчик`, `Доставна цена`

```

```

1      `, `Доставна цена`, `Налично количество`, `Преустановен`) VALUES ('1', 'Хляб \
2      `, `Доставна цена`, `Налично количество`, `Преустановен`) VALUES ('2', 'Хляб \
3      `, `Доставна цена`, `Налично количество`, `Преустановен`) VALUES ('3', 'Захар
4      `, `Доставна цена`, `Налично количество`, `Преустановен`) VALUES ('4', 'Захар
5      `, `Налично количество`, `Преустановен`) VALUES ('Прясно мляко \'Пасище\'', '
6      `, `Налично количество`, `Преустановен`) VALUES ('Прясно мляко \'Ливади\'', '

```

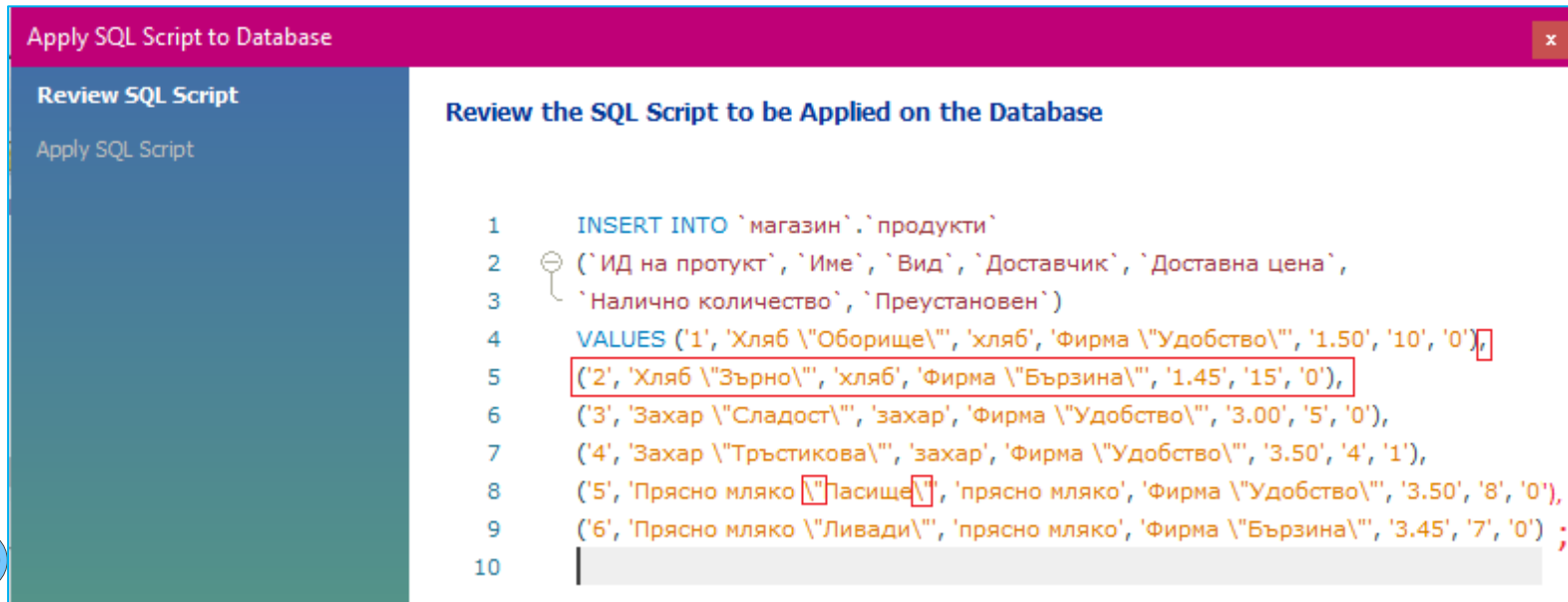
Оглеждаме предложения SQL-код за грешки и възможни оптимизации и **РЕДАКТИРАМЕ** преди да натиснем Apply !!!

```

1      'Оборище\'', 'хляб', 'Фирма \'Удобство\'', '1.50', '10', '0');
2      'Зърно\'', 'хляб', 'Фирма \'Бързина\'', '1.45', '15', '0');
3      'Сладост\'', 'захар', 'Фирма \'Удобство\'', '3.00', '5', '0');
4      'Тръстикова\'', 'захар', 'Фирма \'Удобство\'', '3.50', '4', '1');
5      прясно мляко', 'Фирма \'Удобство\'', '3.50', '8', '0');
6      прясно мляко', 'Фирма \'Бързина\'', '3.45', '7', '0');

```

Редактиране и оптимизиране на SQL- кода



```
1  INSERT INTO `магазин`.`продукти`
2  (`ИД на продукт`, `Име`, `Вид`, `Доставчик`, `Доставна цена`,
3  `Налично количество`, `Преустановен`)
4  VALUES ('1', 'Хляб \"Оборище\"', 'хляб', 'Фирма \"Удобство\"', '1.50', '10', '0'),
5  ('2', 'Хляб \"Зърно\"', 'хляб', 'Фирма \"Бързина\"', '1.45', '15', '0'),
6  ('3', 'Захар \"Сладост\"', 'захар', 'Фирма \"Удобство\"', '3.00', '5', '0'),
7  ('4', 'Захар \"Тръстикова\"', 'захар', 'Фирма \"Удобство\"', '3.50', '4', '1'),
8  ('5', 'Прясно мляко \"Пасище\"', 'прясно мляко', 'Фирма \"Удобство\"', '3.50', '8', '0'),
9  ('6', 'Прясно мляко \"Ливади\"', 'прясно мляко', 'Фирма \"Бързина\"', '3.45', '7', '0') ;
10
```

Забелязахме зависимости, които са много сходни със C# във VS – всеки нов запис на таблицата е между `()`, разделен със `,` а при текст с включени кавички, слагаме наклонена черта за изключване `\`. Краят на кода приключва с `;`;

Най-важното - **DRY (Don't Repeat Yourself)** – да няма повтарящ се код !!!

2 – Таблица „Служители“

Table Name: Schema:

Charset/Collation: Engine:

Comments:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G
ИД на служител	INT	☒	☒	☐	☐	☐	☐	☐	☐
Име	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐
Фамилия	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐
Длъжност	VARCHAR(45)	☐	☐	☐	☐	☐	☐	☐	☐
ЕГН	VARCHAR(10)	☐	☐	☐	☐	☐	☐	☐	☐
Дата на постъпване	DATETIME	☐	☐	☐	☐	☐	☐	☐	☐
Телефон	VARCHAR(10)	☐	☒	☐	☐	☐	☐	☐	☐
Адрес	VARCHAR(45)	☐	☐	☐	☐	☐	☐	☐	☐
E-mail	VARCHAR(45)	☐	☐	☐	☐	☐	☐	☐	☐

```
1 CREATE TABLE `магазин`.`служители` (  
2     `ИД на служител` INT NOT NULL,  
3     `Име` VARCHAR(45) NOT NULL,  
4     `Фамилия` VARCHAR(45) NOT NULL,  
5     `Длъжност` VARCHAR(45) NULL,  
6     `ЕГН` VARCHAR(10) NULL,  
7     `Дата на постъпване` DATETIME NULL,  
8     `Телефон` VARCHAR(10) NOT NULL,  
9     `Адрес` VARCHAR(45) NULL,  
10    `E-mail` VARCHAR(45) NULL,  
11    PRIMARY KEY (`ИД на служител`));
```

Може да се добави и снимка – тип BLOB



Table Name:

Schema: **магазин**

Charset/Collation:

Default Charset

Default Collation

Engine:

InnoDB

Comments:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
 ИД на доставчици	INT	☒	☒	☐	☐	☐	☐	☐	☐	
 Име на фирмата	VARCHAR(45)	☐	☐	☐	☐	☐	☐	☐	☐	
 Лице за контакт	VARCHAR(45)	☐	☐	☐	☐	☐	☐	☐	☐	
 Длъжност	VARCHAR(45)	☐	☐	☐	☐	☐	☐	☐	☐	
 Адрес	VARCHAR(45)	☐	☐	☐	☐	☐	☐	☐	☐	
 Телефон	VARCHAR(10)	☐	☒	☐	☐	☐	☐	☐	☐	
 E-mail	VARCHAR(45)	☐	☐	☐	☐	☐	☐	☐	☐	

Column Name:

Charset/Collation:

Default Charset

Default Collation

Comments:

Data Type:

Default:

Storage:

Move Up

Move Down

Copy

Cut

Paste

Delete Selected

Refresh

Clear Default

Default NULL

Default "

3 – Таблица изцяло с SQL

Apply SQL Script to Database

Review SQL Script

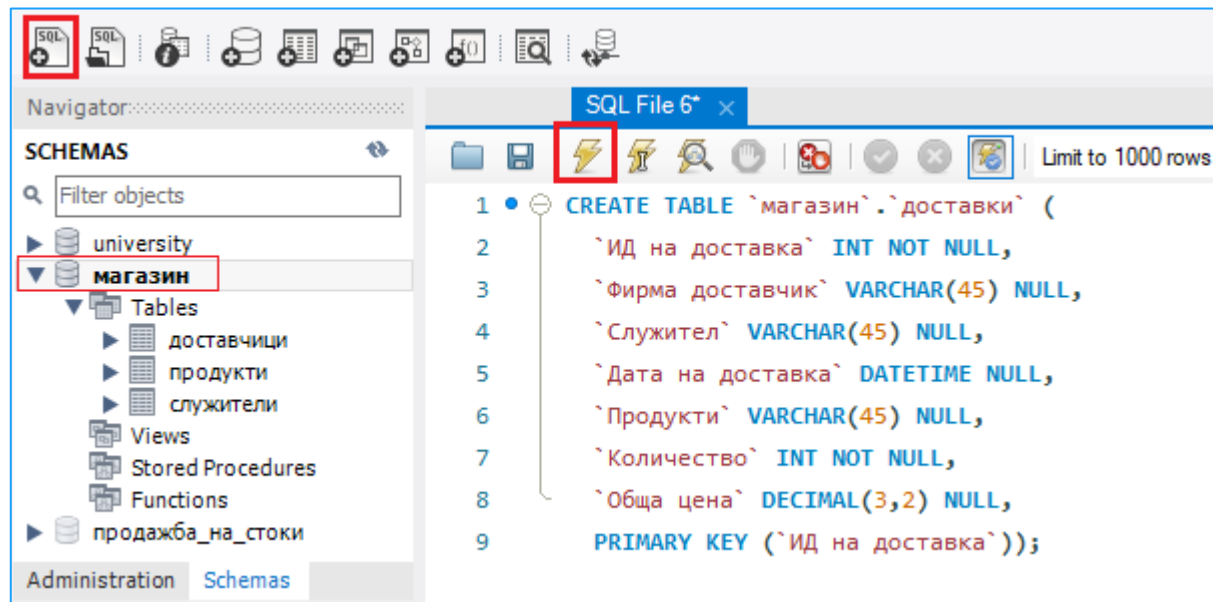
Apply SQL Script

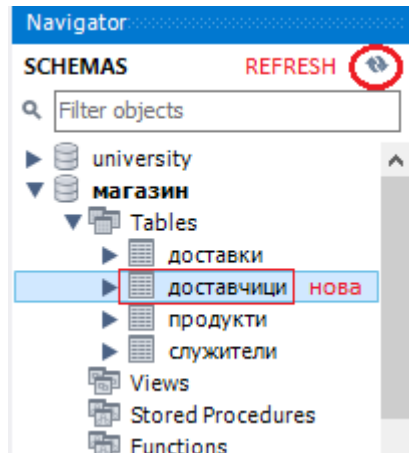
Review the SQL Script to be Applied on the Database

```
1 CREATE TABLE `магазин`.`доставчици` (  
2   `ИД на доставчици` INT NOT NULL,  
3   `Име на фирмата` VARCHAR(45) NULL,  
4   `Лице за контакт` VARCHAR(45) NULL,  
5   `Длъжност` VARCHAR(45) NULL,  
6   `Адрес` VARCHAR(45) NULL,  
7   `Телефон` VARCHAR(10) NOT NULL,  
8   `E-mail` VARCHAR(45) NULL,  
9   PRIMARY KEY (`ИД на доставчици`));  
10
```

Нека копираме предложения SQL-код от втората таблица и го модифицираме така, че да създадем аналогичната трета таблица „Доставки“.

Първата изцяло създадена на SQL-таблица

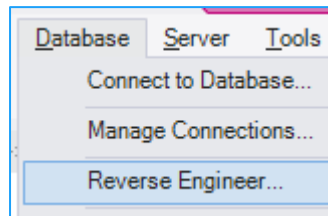




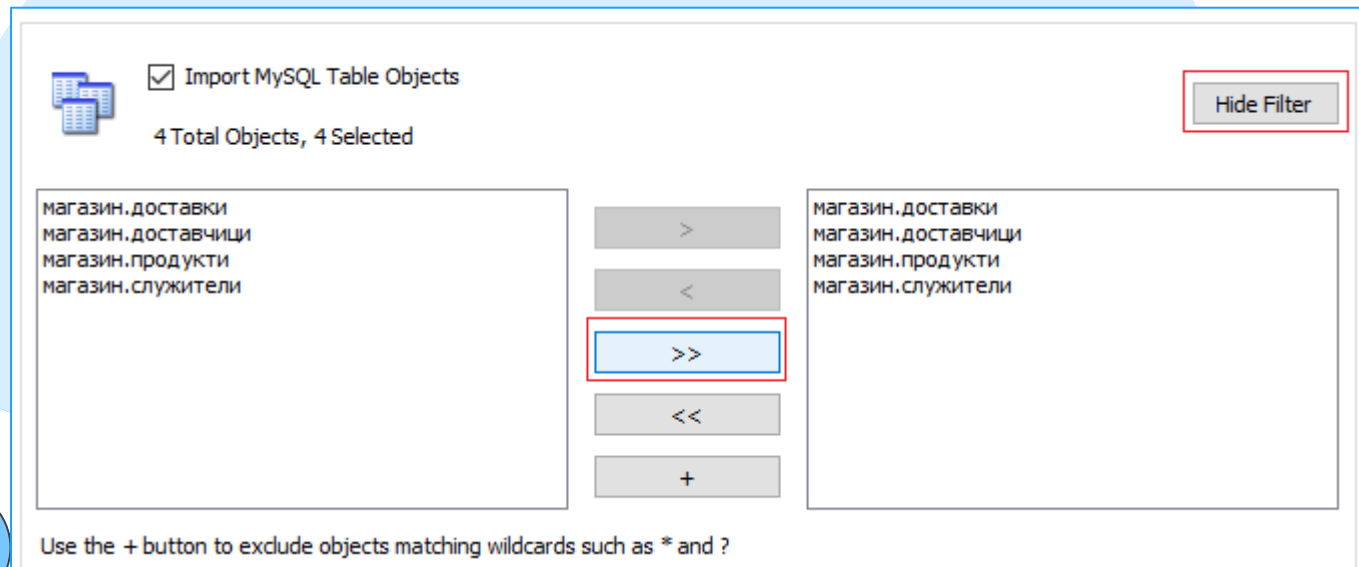
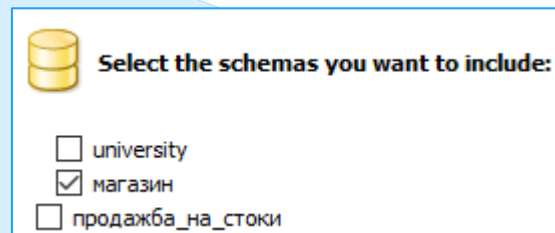
Запомнете, че след всяка промяна, трябва да се обновява изгледа, защото при БД има забавяне във визуализацията. За да се покаже на екрана новосъздадената таблица, натискаме Refresh. Подобно на VS, Можем да проследим изходите / одобряването на резултатите, но трябва да имаме предвид, че хронологията е последователна и най-новият изход е последен надолу, така че трябва да го разлистим. Неодобряване на действие не винаги показва проблем – просто четем съобщението и според него коригираме.

Output				
Action Output				
#	Time	Action	Message	Duration / I
✗ 33	11:55:56	Apply changes to Продукти		
✓ 34	11:57:53	Apply changes to продукти	Changes applied	
✓ 35	11:59:21	SELECT * FROM магазин.продукти LIMIT 0, 1000	0 row(s) returned	0.000 sec /
✗ 36	12:14:10	INSERT INTO 'магазин'. 'продукти' ('ИД на продукт', 'Име', 'Вид', 'Доставчик', 'Дост...	1136: Column count doesn't match value count at row 1	
✓ 37	12:47:26	Apply changes to служители	Changes applied	
✓ 38	12:56:30	Apply changes to доставчици	Changes applied	
✓ 39	13:24:31	CREATE TABLE 'магазин'. 'доставки' ('ИД на доставка' INT NOT NULL, 'Фирма до...	0 row(s) affected	0.031 sec
✗ 40	13:24:37	CREATE TABLE 'магазин'. 'доставки' ('ИД на доставка' INT NOT NULL, 'Фирма до...	Error Code: 1050. Table 'доставки' already exists	0.000 sec

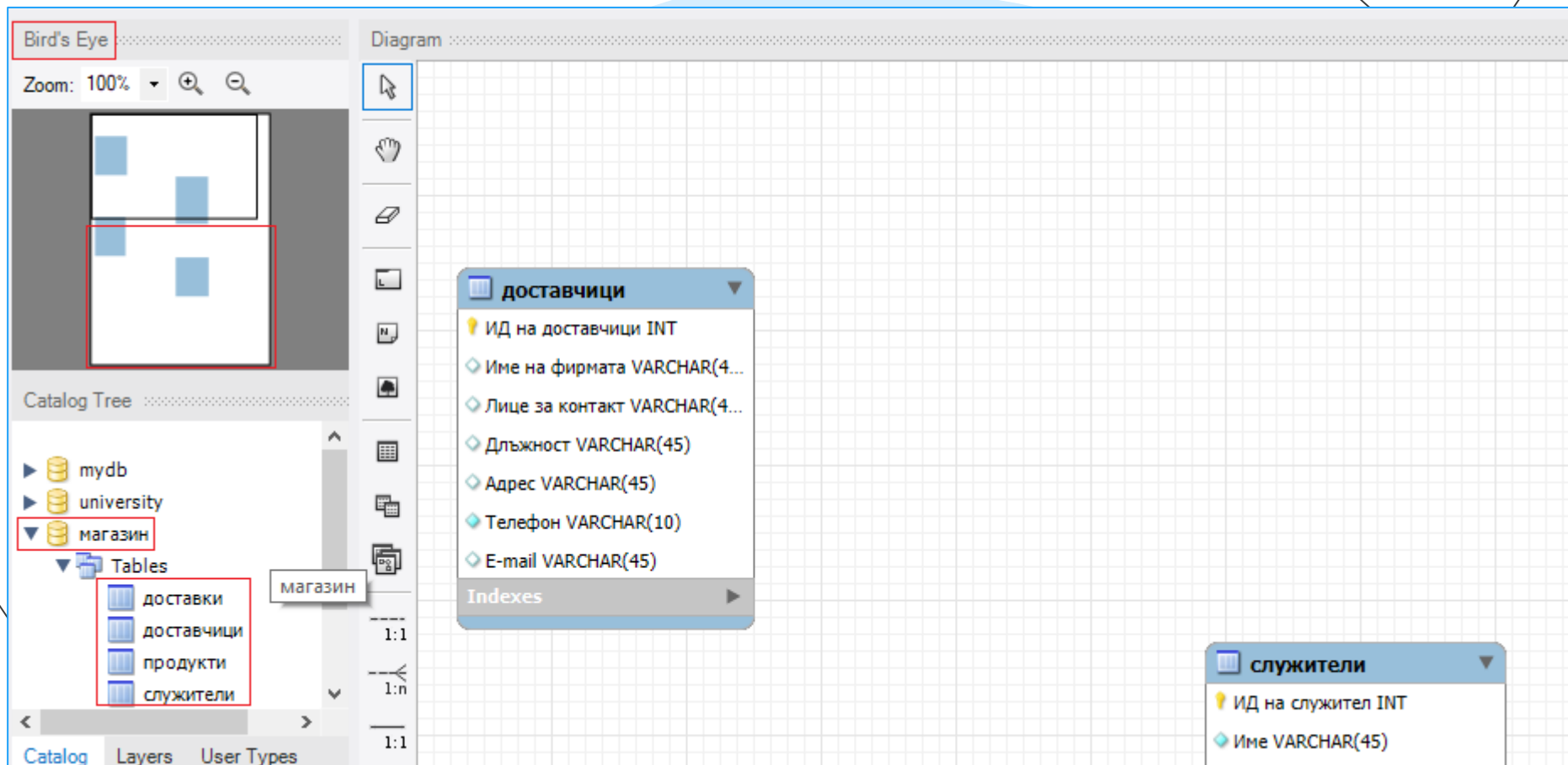
ER-диаграма



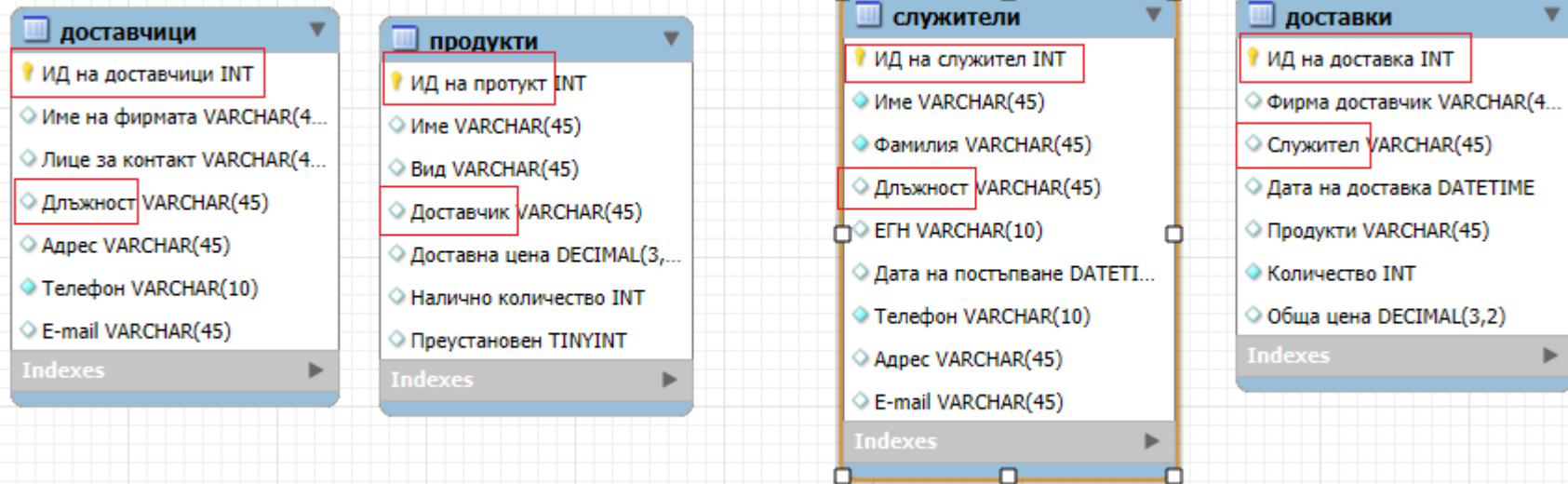
Задаваме името на конекцията, която използваме и БД, която използваме.



Обикновено таблиците са пръснати върху диаграмата и не всички са във видимата зона, а под нея, което може да се прецени от страничния прозорец „Птиче око“.



Типове ключове и релации между тях



Това е черновата на диаграмата, която се получи, т.к. още не сме учили за видовете ключове – първичен – primary key и външен foreign key.

Забелязваме голяма повтораемост в атрибутите на таблиците, затова започваме да мислим за оптимизиране и подобряване в таблиците – например в доставки – вместо **Служител** тип текст, ще го променим на **ИД на служител** INT, като двете таблици ще се свържат по този начин и промяната ще се отрази като външен ключ.

Промяна на атрибут с цел връзка между 2 таблици

Table Name: Schema: **магазин**

Charset/Collation: Engine:

Comments:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Extra
ИД на доставка	INT	☒	☒	☐	☐	☐	☐	☐	☐	
Фирма доставчик	VARCHAR(45)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
ИД на служител	INT	☐	☐	☐	☐	☐	☐	☐	☐	NULL
Дата на доставка	DATETIME	☐	☐	☐	☐	☐	☐	☐	☐	NULL
Продукти	VARCHAR(45)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
Количество	INT	☐	☒	☐	☐	☐	☐	☐	☐	
Обща цена	DECIMAL(3,2)	☐	☐	☐	☐	☐	☐	☐	☐	NULL

Единият начин за осъществяване на тази промяна е с връщане в основния таб и създаване на нова диаграма с повтаряне на предходните стъпки за създаване на нова диаграма.

```
1 ALTER TABLE `магазин`.`доставки`  
2 CHANGE COLUMN `Служител` `ИД на служител` INT NULL DEFAULT NULL ;  
3
```

Трябва да има пълно съвпадение между името и типа на атрибутите в двете таблици, за да могат да се свържат. Отново прочитаме SQL-кода.

Първичен и Външен Ключ

Diagram

доставчици

- ИД на доставчици INT
- Име на фирмата VARCHAR(45)
- Лице за контакт VARCHAR(45)
- Длъжност VARCHAR(45)
- Адрес VARCHAR(45)

служители

- ИД на служител INT
- Име VARCHAR(45)
- Фамилия VARCHAR(45)
- Длъжност VARCHAR(45)
- ЕГН VARCHAR(10)

доставки

- ИД на доставка INT
- Фирма доставчик VARCHAR(45)
- ИД на служител INT
- Дата на доставка DATETIME
- Продукти VARCHAR(45)

продукти

- ИД на продукт INT
- Име VARCHAR(45)
- Вид VARCHAR(45)
- Доставчик VARCHAR(45)
- Доставна цена DECIMAL(3, ...)

доставки - Table

Table Name: доставки Schema: магазин

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
ИД на доставка	INT	☒	☒	☐	☐	☐	☐	☐	☐	
Фирма доставчик	VARCHAR(45)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
ИД на служител	INT	☐	☐	☐	☐	☐	☐	☐	☐	NULL
Дата на доставка	DATETIME	☐	☐	☐	☐	☐	☐	☐	☐	NULL

Column Name: ИД на служител Data Type: INT

Charset/Collation: Default Charset Default Collation

Default: NULL

Storage: ☐ Virtual ☐ Stored

☐ Primary Key ☐ Not Null ☐ Unique

☐ Binary ☐ Unsigned ☐ Zero Fill

☐ Auto Increment ☐ Generated

Columns Foreign Keys Triggers Partitioning Options Inserts Privileges

Потребителският интерфейс обикновено предоставя по няколко начина, а другият по-кратък е директно кликуване върху таблицата, която искаме да редактираме.

доставчици

- ИД на доставчици INT
- Име на фирмата VARCHAR(45)
- Лице за контакт VARCHAR(45)
- Длъжност VARCHAR(45)
- Адрес VARCHAR(45)

служители

- ИД на служител INT
- Име VARCHAR(45)
- Фамилия VARCHAR(45)
- Длъжност VARCHAR(45)
- ЕГН VARCHAR(10)

доставки

- ИД на доставка INT
- Фирма доставчик VARCHAR(45)
- ИД на служител INT
- Дата на доставка DATETIME
- Продукти VARCHAR(45)

продукти

- ИД на продукт INT
- Име VARCHAR(45)
- Вид VARCHAR(45)
- Доставчик VARCHAR(45)
- Доставна цена DECIMAL(3,...

доставки - Table

Table Name: доставки Schema: магазин

Foreign Key Name	Referenced Table	Column	Referenced Column
ИД на служител	магазин, служители	☒ ИД на служител	ИД на служител
атрибут	БД . таблица	☐ Дата на доставка	
		☐ Продукти	
		☐ Количество	
		☐ Обща цена	

Foreign Key Options

On Update: NO ACTION

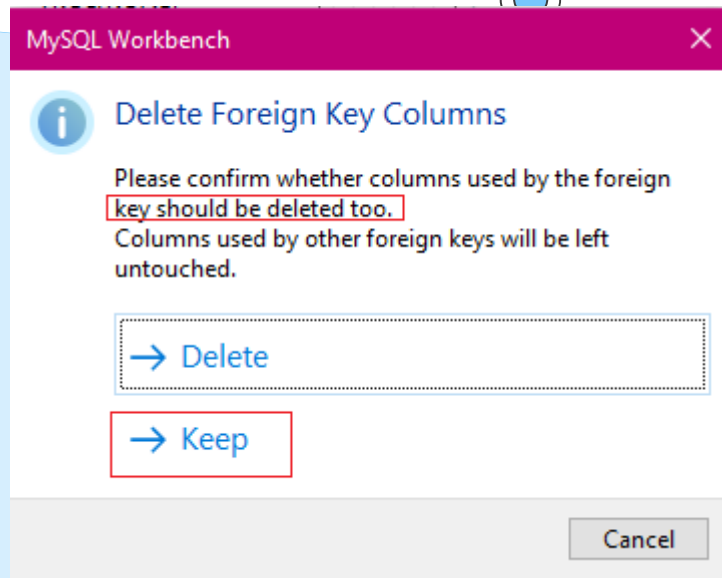
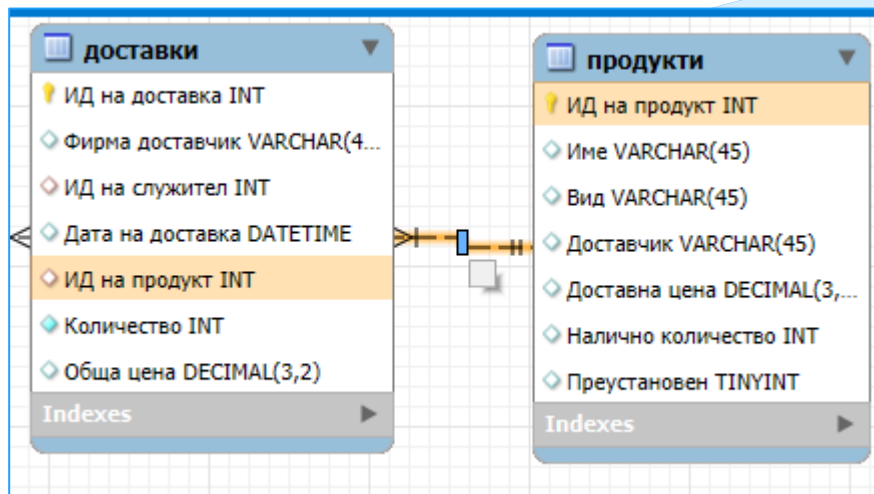
On Delete: NO ACTION

☐ Skip in SQL generation

- Foreign Key Comment

Ако изречно свържем двете таблици чрез външен ключ по съвпадащи атрибути, връзката става пунктирана. При това забелязваме, че тя показва и съотношение 1:n = 1:много = 1 служител може да осъществява много доставки. **Трябва да имаме предвид, че при връзки 1:n, винаги избираме първо таблицата, която представлява много.**

Изтриване на съществуваща връзка:



При БД, ако селектираме връзка – тя показва кои атрибути свързва. При натискане на DEL, не бива инстинктивно да натискам, че сме съгласни, а четем съобщението, защото ще изтрие не само цялата връзка, но и атрибутите ѝ и по този начин, таблиците ще се свият. За да изтрием само връзка, след DEL селектираме Keep.

Релации/връзки между няколко таблици

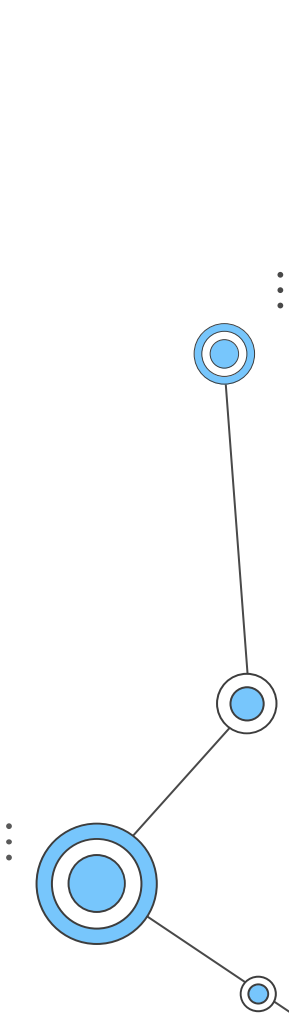
Table Name: доставки			
Column Name	Datatype	PK	NN
ИД на доставка	INT	☒	☒
ИД на доставчик	INT	☐	☐
ИД на служител	INT	☐	☐
Дата на доставка	DATETIME	☐	☐
ИД на продукт	INT	☐	☐

Table Name: продукти			
Column Name	Datatype	PK	NN
ИД на продукт	INT	☒	☒
Име	VARCHAR(45)	☐	☐
Вид	VARCHAR(45)	☐	☐
ИД на доставчик	INT	☐	☐
Доставна цена	DECIMAL(3,2)	☐	☐

```
1 ALTER TABLE `магазин`.`доставки`  
2 CHANGE COLUMN `Фирма доставчик` `ИД на доставчик`  
3 INT NULL DEFAULT NULL ;  
4
```

По време на моделирането на БД е нормално да се правят оптимизации. В случая, преценяваме, че атрибут „Доставчик“ е много по-добре да бъде „ИД на доставчик“ с цел да няма повторение, а директно да свържем 2 таблици чрез ИД и от там да извлечем информацията. Оказва се, че има записи, които трябва да се изтрият, за да бъдат одобрени промените, т.к. в съобщението за грешка текстов тип не може да бъде преобразуван в числов.

ИД на продукт	Име	Вид	Доставчик	Доставна цена	Налично количество	Преустановен
1	Хляб "Оборище"	хляб	изтриваме записите	1.50	10	0
2	Хляб "Зърно"	хляб		1.45	15	0
3	Захар "Сладост"	захар		3.00	5	0
4	Захар "Тръстикова"	захар		3.50	4	1
5	Пряно мляко "Пасище"	пряно мляко		3.50	8	0
6	Пряно мляко "Ливади"	пряно мляко		3.45	7	0



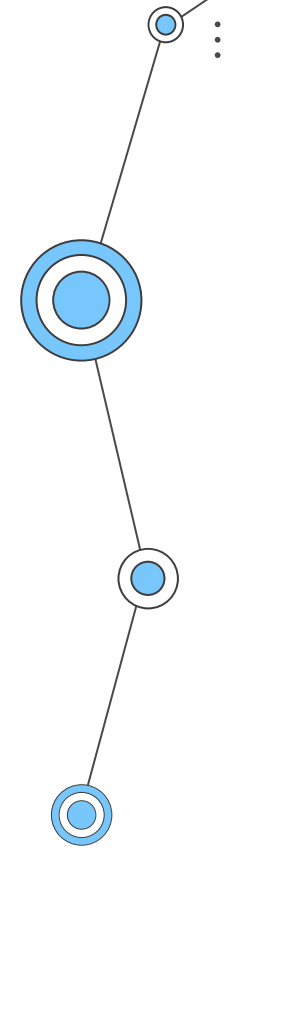
```
1 UPDATE `магазин`.`продукти` SET `Доставчик` = ""
2 WHERE (`ИД на протукт` = '1');
3 UPDATE `магазин`.`продукти` SET `Доставчик` = ""
4 WHERE (`ИД на протукт` = '2');
5 UPDATE `магазин`.`продукти` SET `Доставчик` = ""
6 WHERE (`ИД на протукт` = '3');
7 UPDATE `магазин`.`продукти` SET `Доставчик` = ""
8 WHERE (`ИД на протукт` = '4');
9 UPDATE `магазин`.`продукти` SET `Доставчик` = ""
10 WHERE (`ИД на протукт` = '5');
11 UPDATE `магазин`.`продукти` SET `Доставчик` = ""
12 WHERE (`ИД на протукт` = '6');
13
```

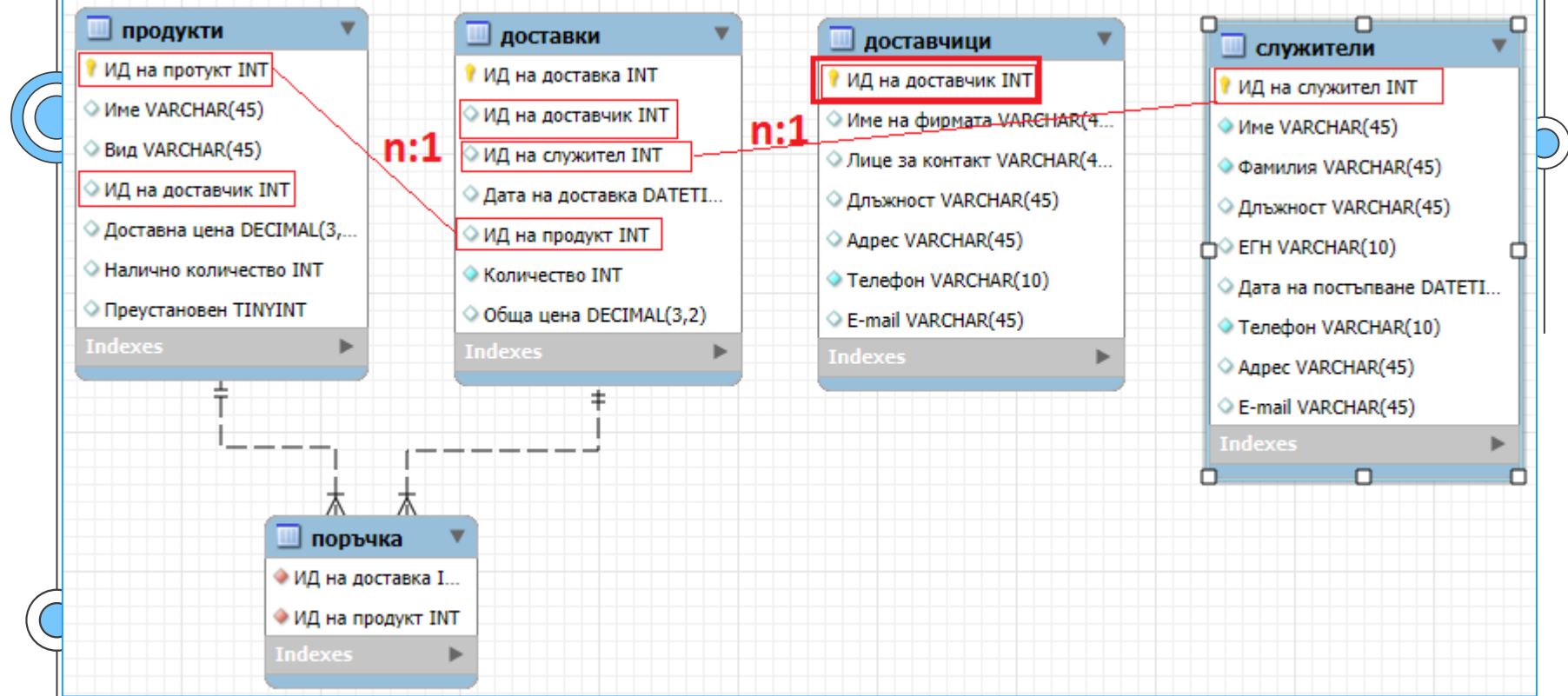
Друг начин е директно да се изтрие цялата колона

```
1 ALTER TABLE `магазин`.`продукти` DROP COLUMN `Доставчик`;
2
```

и после да се вмъкне от настройките наново (Д.б. Move Up):

```
1 ALTER TABLE `магазин`.`продукти`
2 ADD COLUMN `ИД на доставчик` INT NULL AFTER `Вид`;
3
```





Когато проверим диаграмата, нашата БД вече има много по-ясен и подреден вид. Остава да направим връзките между таблиците, като изберем подходящи описателни имена и ги свържем с повтарящите се атрибути (ИД на...), които ще станат външни ключове. Обръщам внимание, че ИД на доставчици с първичния ключ в таблица Доставчици трябва да се свърже с другите две.

Промяна на атрибут с цел връзка между 2 таблици

```
1 ALTER TABLE `магазин`.`продукти`
2 CHANGE COLUMN `ИД на протукт` `ИД на продукт` INT NOT NULL ;
3 ALTER TABLE `магазин`.`продукти`
4 ADD CONSTRAINT `ВК_продукти-доставки`
5 FOREIGN KEY (`ИД на продукт`)
6 REFERENCES `магазин`.`продукти` (`ИД на продукт`)
7 ON DELETE NO ACTION
8 ON UPDATE NO ACTION;
9
```

Тази промяна, колкото и да изглежда правилна за връзка между 2 таблици, не е и ще бъде отхвърлена, защото таблицата с ВК обикновено е в релация 1: много с таблицата с ПК, а правилото е, че винаги се започва с таблицата, която е в позицията на МНОГО.

PK = Primary Key
ПК = Първичен Ключ

FK = Foreign Key
ВК = Външен Ключ

Table Name: доставки		Schema: магазин	
Foreign Key Name	Referenced Table	Column	Referenced Column
ВК_доставки_продукти	`магазин`.`продукти`	☐ ИД на доставка	
Описателно име - ВК = Външен Ключ FN = Foreign Key и имената на 2-те таблици		☐ ИД на доставчи...	
		☐ ИД на служител	
		☐ Дата на доставка	
		☒ ИД на продукт	ИД на продукт
		☐ Количество	ИД на продукт
		☐ Обща цена	Име
			Вид
			ИД на доставчик
			Доставна цена
			Налично количество
			Преустановен
			Specify Column...



Table Name: **продукти** n, FK Schema: **магазин**

Foreign Key Name	Referenced Table
ВК_доставчици_продукти	`магазин`.`доставчици`
1: n	1, PK

Column Referenced Column

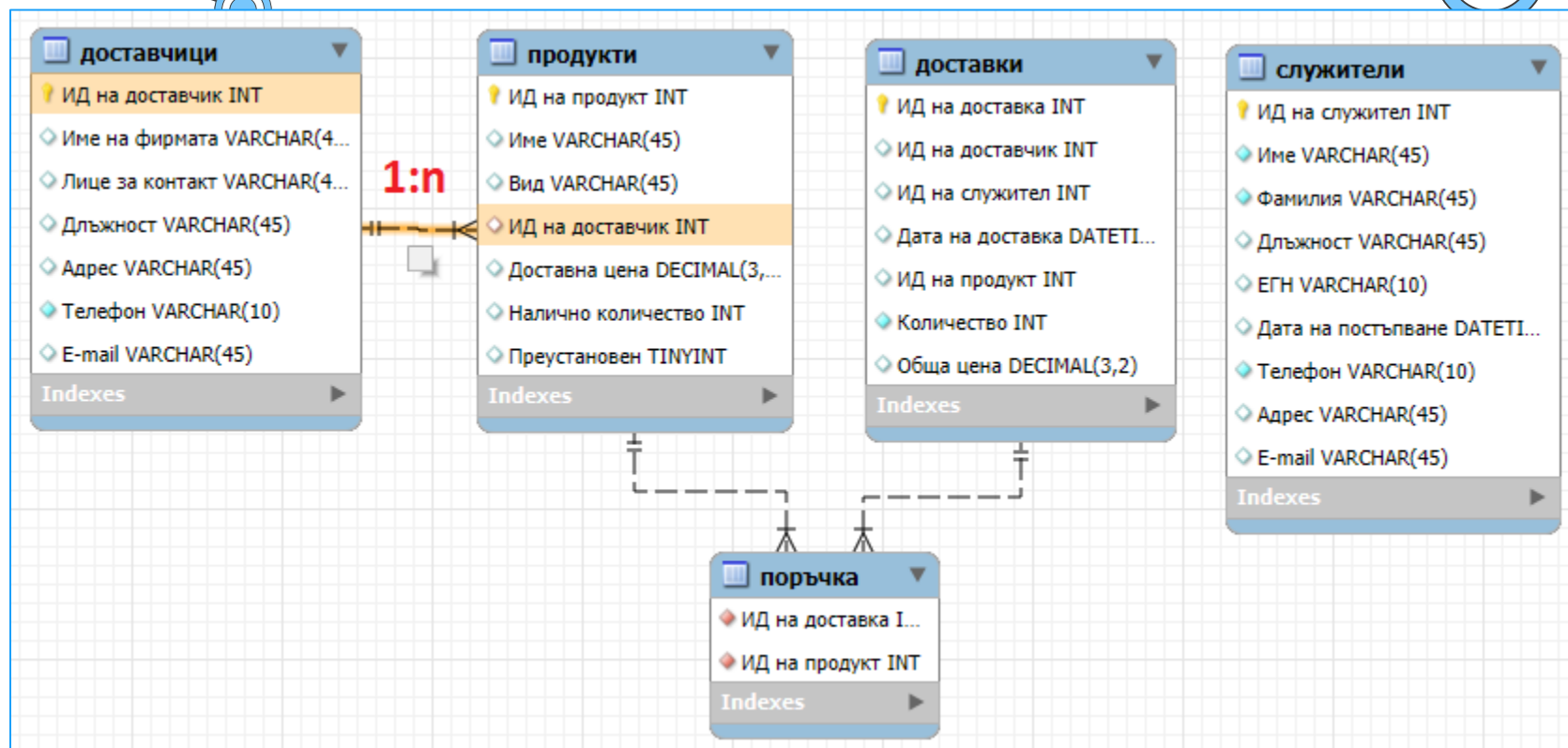
☐ ИД на продукт	
☐ Име	
☐ Вид	
☒ ИД на доставчик	ИД на доставчик
☐ Доставка цена	
☐ Налично колич...	
☐ Преустановен	

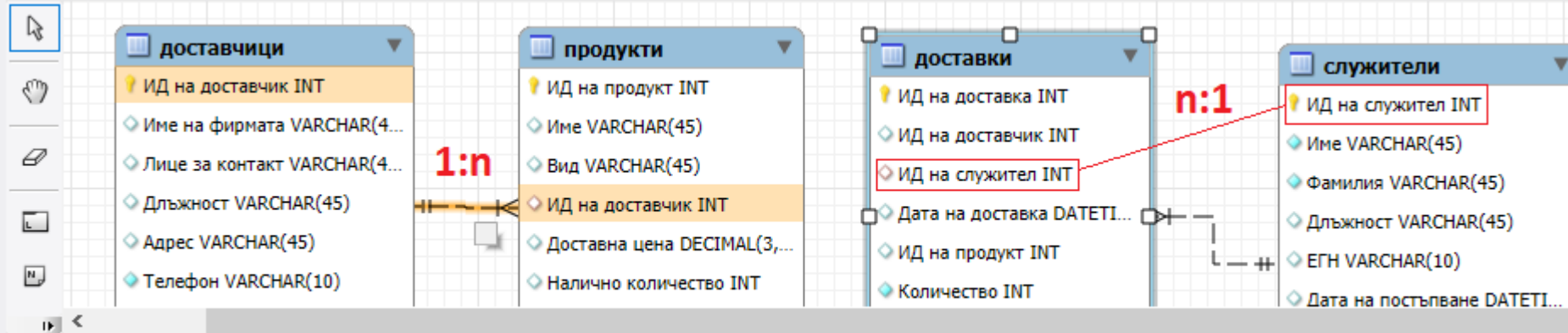
Четете!!!

Columns Indexes **Foreign Keys** Triggers Partitioning Options

```
1 ALTER TABLE `магазин`.`продукти`  
2 ADD INDEX `ВК_доставчици_продукти_idx` (`ИД на доставчик` ASC) VISIBLE;  
3 ;  
4 ALTER TABLE `магазин`.`продукти`  
5 ADD CONSTRAINT `ВК_доставчици_продукти`  
6 FOREIGN KEY (`ИД на доставчик`)  
7 REFERENCES `магазин`.`доставчици` (`ИД на доставчик`)  
8 ON DELETE NO ACTION  
9 ON UPDATE NO ACTION;  
10
```

Резултат до момента:





доставки - Table

Table Name: доставки **n, FK** Schema: магазин

Charset/Collation: utf8mb4 utf8mb4_0900_ai_ci Engine: InnoDB

Comments:

Foreign Key Name	Referenced Table	Column	Referenced Column
BK_доставки_служители	магазин, служители	☐ ИД на доставка	
		☐ ИД на доставчик	
		☒ ИД на служител	ИД на служител
		☐ Дата на доставка	

Foreign Key Options

On Update: NO ACTION

On Delete: NO ACTION

Foreign Key Comment



Аналогично свързваме двете таблици Доставки-Служители, като винаги започваме от таблицата с ВК, която представлява МНОГО

Винаги маркираме и започваме работа първо от таблицата за МНОГО с Външния ключ и нея свързваме с таблицата с 1 с Първичния Ключ

доставчици

- ИД на доставчик INT
- Име на фирмата VARCHAR(45)
- Лице за контакт VARCHAR(45)
- Длъжност VARCHAR(45)
- Адрес VARCHAR(45)
- Телефон VARCHAR(10)

продукти

- ИД на продукт INT
- Име VARCHAR(45)
- Вид VARCHAR(45)
- ИД на доставчик INT
- Доставна цена DECIMAL(3,2)
- Налично количество INT

доставки

- ИД на доставка INT
- ИД на доставчик INT
- ИД на служител INT
- Дата на доставка DATETIME
- ИД на продукт INT
- Количество INT

служители

- ИД на служител INT
- Име VARCHAR(45)
- Фамилия VARCHAR(45)
- Длъжност VARCHAR(45)
- ЕГН VARCHAR(10)
- Дата на постъпване DATETIME

доставки - Table

Table Name: Schema: **магазин**

Foreign Key Name	Referenced Table
BK_доставки_служители	`магазин`.`служители`
BK_доставки_продукти	`магазин`.`продукти`

Column	Referenced Column
☐ ИД на доставка	
☐ ИД на доставчик	
☐ ИД на служител	
☐ Дата на доставка	
☒ ИД на продукт	ИД на продукт
☐ Количество	
☐ Обща цена	

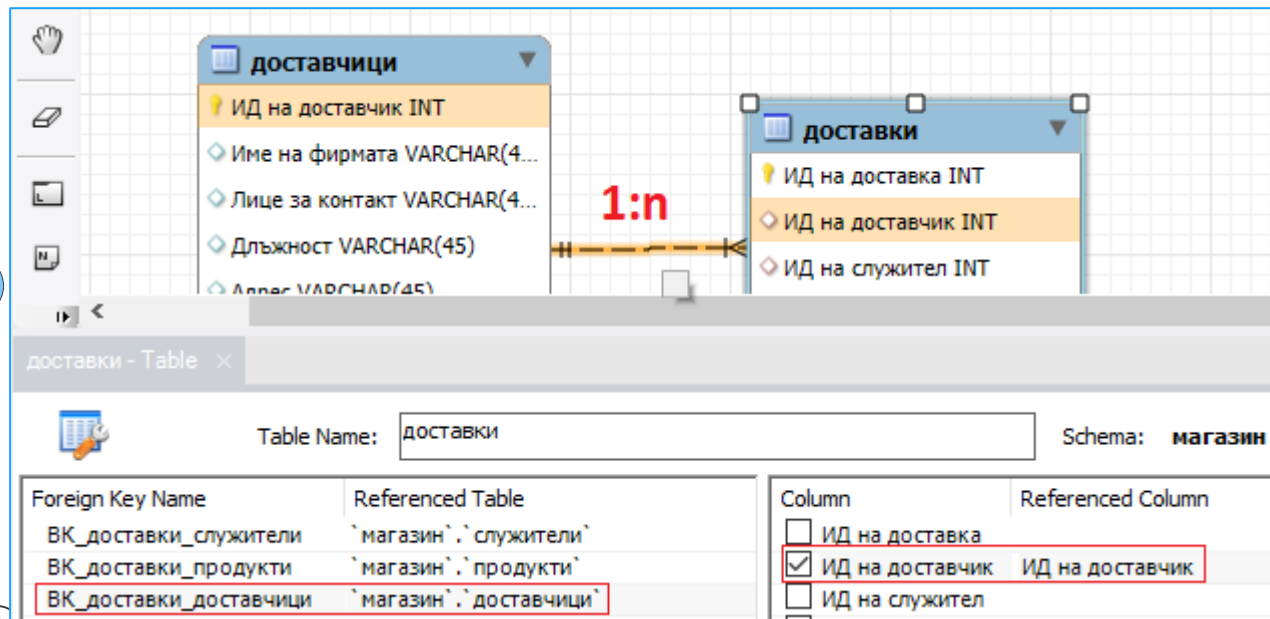
Foreign Key Options

On Update: **NO ACTION**

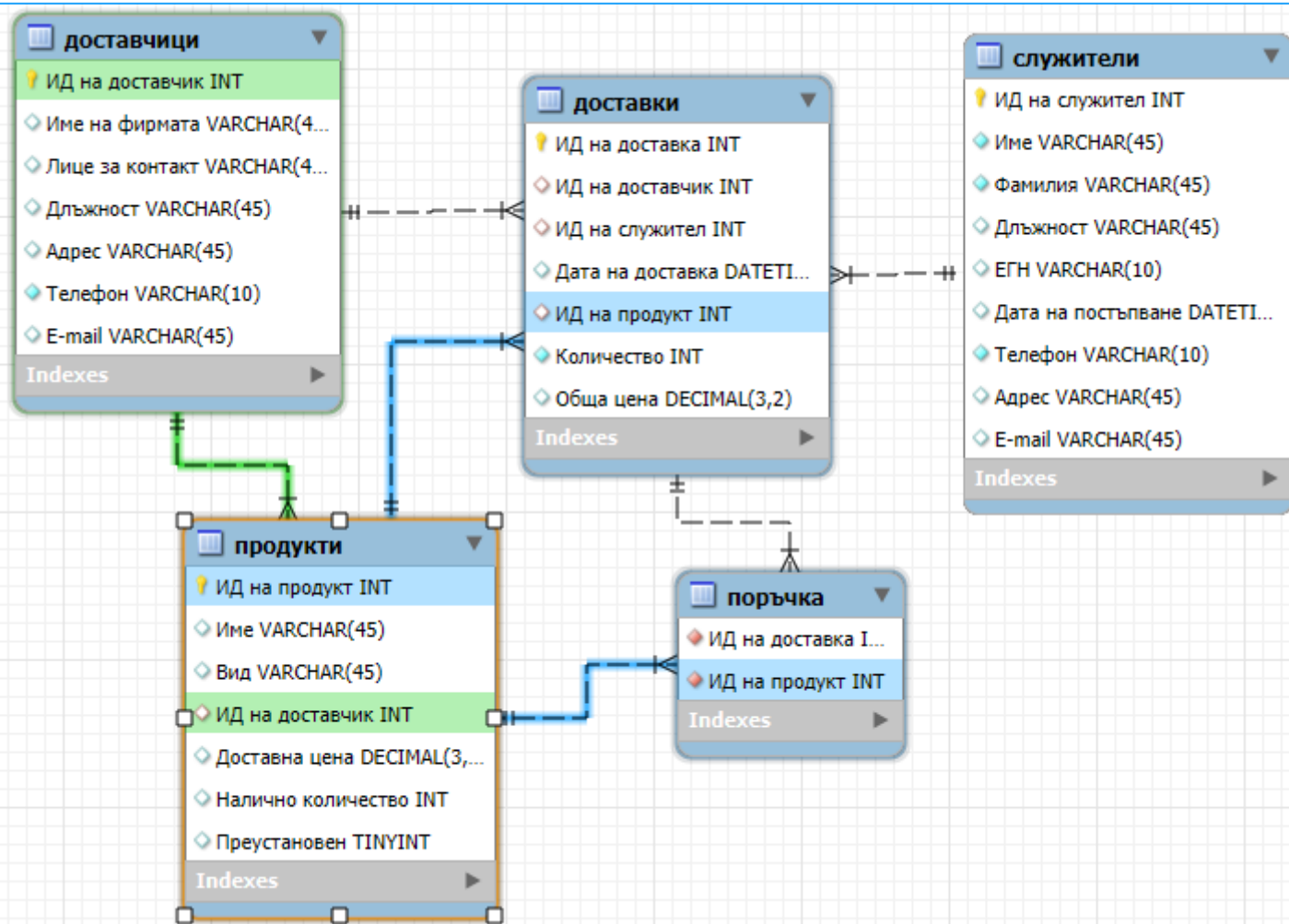
On Delete: **NO ACTION**

☐ Skip in SQL generation

Foreign Key Comment



Вече диаграмата е готова, можем да я погледнем в завършен вид и попълваме с данни, за да може после те да бъдат обработени чрез SQL заявки.



Благодаря!

